University of Cape Town
Department of Computer Science

Computer Science CSC3003S
Class Test 1, 2025

Answer all questions.
Approximate marks per question are shown in brackets.

This test has 6 pages.

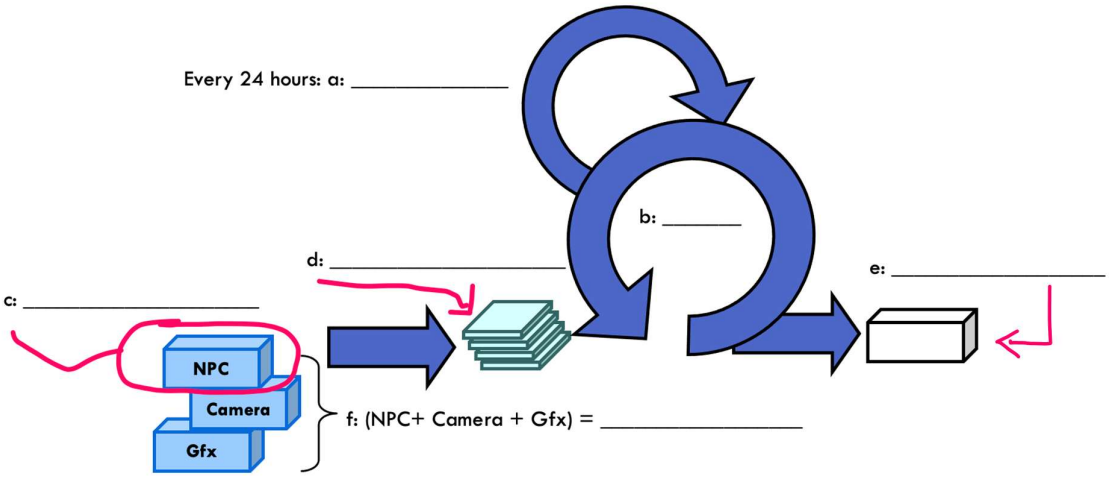
Time: 45 minutes **Marks: 35**

Question 1 [8]

a. Explain the difference between a "process" and a "project" [2]
*A process is a **routine** that gets **repeated** daily/weekly. [1] This makes it **controlled**. It can be **refined each iteration** to satisfy all three of the **better+faster+cheaper** quality aspects.*
*A project is **not routine**. [1] It **never repeats** the same thing. This makes it **hard to manage (complex/chaotic)**. You can only satisfy **two out of better+faster+cheaper***
Process good, project bad [0.5]
Process is well known, project is unknown [0.5]

b. In his book, The Mythical Man Month, Brooks explains how adding more developers to an overdue project delays it even more, why is this? [2]
*Late in the development stage, the existing developers have learned much out the project, how to program it and have had to solve many problems. Adding new developers means: A: increasing the number of communication channels ½(n*n-1) communication lines. B: new developers have to spend time learning*
C: old developers have to spend time training new developers
D: roles are now unclear, who does what?
E: Every additional developer added to a project (whether late or not) add a decreasing amount of productivity gain, beyond a certain number the gain is negative.

c. Consider the below diagram of the Scrum methodology.
Add four out of six missing labels, the scribbles point to the part to be named. [4]
Write down at least 4 out of 6 labels, out of (a) to (f), in your answer book.
Make sure to write the letter next to your label.
a: daily scrum (meeting) / daily standup (meeting) / daily 15 min meeting
b: sprint
c: sprint backlog / product backlog
d: (sprint) backlog tasks (selected by team)
e: iteration/increment/definition of done/shippable product
f: product backlog


Every 24 hours: a: _____
b: _____
d: _____
e: _____
c: _____
NPC
Camera
Gfx
f: (NPC+ Camera + Gfx) = _____

Refer to the text below and answer the questions relating to it:

As discussed in the lectures, MCAS (Manoeuvring Characteristics Augmentation System) is a software system on the Boeing 737 Max family of planes that can alter the pitch (nose up/down) of the plane to make it behave and feel exactly like the older Boeing 737 NG planes. By making the new 737 Max feel the same as the old 737 NG plane, Boeing customers did not have to train their pilots for the new Max, but could have their pilots use their existing training for older 737 types instead, thus saving customers money.

However as designed the MCAS system was too powerful, and after MCAS caused two plane crashes, the FAA calculated that if not redesigned the MCAS system would have crashed 15 planes over its lifetime and killed over 2000 passengers. In reality, it crashed 2 planes in a 5 month timespan and killed 346 people. From a financial point of view, Boeing suffered ~$ 20 Billion in direct costs and ~$ 60 Billion in lost sales (which may – or may not – be recouped in the future).

Question 2 [7]

a. *You might very well think that Boeing's MCAS software development team did not perform a risk analysis prior to deploying their final MCAS prototype in the 737 Max planes sold to customers.*
Devise a risk strategy and based on that, advise Boeing management on what actions to take to manage the risk associated with this deployment. [2]
Risk strategy: avoid. Advice: do not implement MCAS as designed, fess up to the FAA and inform pilots and implement a much simpler system that can trivially be overruled by pilots. This may mean that pilots will have to be retrained.

b. *The MCAS software development team spend considerable effort designing the MCAS features such that neither the aviation oversight board (the FAA), nor the pilots flying the 737 Max planes were aware of its existence and its true power.*
In so doing, which ISO 25010-2011 quality aspect did the developers ignore? Name the quality aspect and explain how its improves software. [2]
Quality aspect: safety (do not harm humans), explain: it ensures that software does not cause harm to humans (death or injuries or disease)

c. Explain what a "*wicked problem*" is. Are the dual crashes of Lion Air and Ethiopian Airlines caused by the MCAS system due to a "*wicked problem*" Explain why or why not. [3]
A wicked problem is an "unknown unknown". A problem you did not/could not know even existed (i.e.: always an accidental problem). You also do not/cannot know how to solve it without extensive study/research/modelling.
MCAS is NOT a wicked problem, it was easy to foresee that pilots will not be able to debug a system that they do not know exists. The Ethiopian Airline pilots did know the system existed and they did know how to turn it off, but they still could not help crash the plane, because turning off MCAS also turned off the only system that could have saved their plane.
The developers encountered this same problem during testing in the simulator, and crashed the simulated plane but laughed it off and noted that test-pilots (super humans who know every detail of the plane inside-out) did not have problems.

Question 3 [3]

What is aspect oriented programming (AOP) and how does it solve crosscutting concerns? You do not get points for merely mentioning the words: 'pointcut', 'advice' and 'aspect', instead explain how it works. [3]

AOP instruments the source code with (or injects into the object code)[1] an advice and a pointcut. The aspect is the combination of an advice and a pointcut.[0.5] The advice is source code that is handed the parameters of the function that it instruments. [0.5]
The pointcut is one or more locations where the advice gets injected. [0.5]
This pointcut is expressed as a regex (wildcards/string matching) that matches an object + function signature [0.5]. The pointcut can add the advice code before, after, or around the instrumented function [1]. Because AOP injects a single advice (piece of source code) in different locations it promotes DRY code [0.5]. AOP uses a separate compilation step called a weaving (or Aspect) compiler that added the advice source code in the pointcut locations [1].
AOP is used to add security checks/logging after the fact [0.5]

Question 4 [4]

Consider the following lambda:

```
final var a = new ObjectX();
var my_lambda = (int i)-> {
    a.doStuff(i);
};
```

Write a class equivalent to this lambda in either Java or C++ (whichever one you are most comfortable with; note: the syntax does not have to match 100%), but do include all needed method(s), data member(s) and all needed visibility specifier(s). [4]

```
class my_lambda final[1] {
private:[1] //implied by class keyword in C++, needed in Java
  ObjectX captured_a;[1] //can have any name
  //capture by value, must make a copy
  //private constructor, every lambda is a singleton
  my_lambda(ObjectX a): ObjectX(a.clone());[1]
public:[1]
  void operator()(int i) {[1] //any name is fine
    //VV this code must be present for point above
    captured_a.doStuff(i); //must link to captured copy above
  }
}
```

Code for question 5 on the next page.

```
[...] //includes
class Pear {
  int weight;
public:
  Pear(const int weight): weight(weight) {}
  int juice() const { return weight / 2; }
  void takeJuice(const int amount) { weight -= (amount * 2); }
};
class Grapes {
  int count;
public:
  Grapes(const int count): count(count) {}
  int wine() const { return count * 2; }
  void takeWine(const int amount) { count -= (amount / 2); }
};

using Fruit = std::variant<Pear, Grapes>;

class Print {
public:
  void operator()(const Pear& p) const { //worker method
    std::println("Hmm juicy pear {}", p.juice());
  }
  void operator()(const Grapes& g) const {
    std::println("{} grapes hold {} wine", g.count(), g.wine());
  }
};
class Squeeze {
    const int amount;
public:
    Squeeze(const int amount): amount(amount) {}
    void operator()(Pear& p) const { p.takeJuice(amount); }
    void operator()(Grapes& g) const { g.takeWine(amount); }
};

void PrintAll(const std::vector<Fruit>& fruits) {
    for (auto& f: fruits) { std::applyPattern( Print(), f); }
}

void SqueezeAll(std::vector<Fruit>& fruits, const int amount) {
    for (auto& f: fruits) { std::applyPattern( Squeeze(amount), f);}
}

int main() {
    std::vector<Fruit> fruits;
    fruits.push_back( Pear{10} );
    fruits.push_back( Grapes{20} );
    fruits.push_back( Grapes{30} );
    PrintAll(fruits);
    SqueezeAll(fruits, 1);
    PrintAll(fruits);
}
```

Question 5 [7]

Consider the code on the previous page.

a. *The std::applyPattern is not the real name of the method used in this pattern.* Which OOP design pattern does this code model? Explain what problem this pattern solves. [3]

The (acyclic/improved[0.5]) visitor pattern [0.5]. This pattern makes it easy to add both[1] objects and operations to a system. The above improved version eliminates dependencies and cycles between objects and operations[1]. There is no need to a common ancestor/interface to group objects[1] All objects and operations are standalone[1]. The Fruit variant implements the visit method that is the key to the visitor pattern[1]. This satisfied the SOLID (S and O) principles. (Single responsibility: one class (either type or operation) does one thing. Open Closed: easy to extend, hard to change. [1]

In the above code the `std::variant` is a polymorphic class. As discussed in the lecture it consists of a tag field (in the example tag = tag_circle or tag_square) and space to accommodate one (and only one) of the classes it can model (in the example either `Square` or `Circle`).

b. Name the proper CS term for `std::variant`? (do not use the word 'variant') [1]
tagged union, or ~~variant, variant record~~, choice type, discriminated union, disjoint union, sum type, or coproduct

Notice in the above code how none of the classes use (or need) inheritance.
In this variant (pun intended) of the well-known XXX design pattern this aids 4 of the 5 S.O.L.I.D. principles.

c. For 3 out of those 4 principles, list its full name and explain how that particular SOLID principle is satisfied by the structure of the design pattern in the example code on the previous page. [3]
S: single responsibility[1]: each class only does one thing and classes are standalone
O: Open/closed[1]: classes are easy to change, esp. because they are standalone there is no need to modify the code if you want to add functionality.
D: Dependency inversion [1] (the details are decoupled from the overall pattern)
L: Liskov substitution [-1], this is the one it does not address
I: Interface segregation [1]: clients do not depend on and interface they don't use

Question 6 [6]

In the lectures we've discussed data oriented design (DOD aka DOP).

Redesign the following game pseudo-code such that it adheres to the principles of data oriented design. The syntax does not have to be 100% correct.
If the example code shows '...' your redesigned code should also show ...
Briefly explain the changes you made (how do they improve the code DOD-wise?)
The below code is Java code.

```
//imports
public class Sprite {
  public Integer x, y; //x,y is middle of the sprite
  public Integer h, w; //height, weight
  Bitmap picture; //no animation yet, just movement
  void Move(int speed, int direction) {...}
  boolean Overlaps(Sprite s) {...}
}
abstract class baseZombie extends Sprite {
  public Integer speed;
  public Integer direction; //1..360 degrees
  abstract void display();
  abstract Integer Overlaps(Sprite s); //returns damage on hit
}
//...
many different zombie types: class ZombieX extends baseZombie {...}
//...
public class Player extends Sprite {
  final Integer speed = 100; //fixed speed
  public Integer direction; //1..360 degrees
  public Integer health = 100;
}

public class Game {
  final int maxZombieCount = 10000; //never more than 10'000 zombies
  Sprite player;
  ArrayList<baseZombie> zombies;
  Game() {...}
  void step() {
    player.Move(...);
    for (var z: zombies) { z.Move(...); }
  }
  void doPlayerHit() {
    var oldHealth = player.health;
    for (var z: zombies) { player.health -= z.Overlaps(player); }
    if (player.health != oldHealth) { ... }
  }
  void display() { //runs 30x per second
    doPlayerHit();
    player.display();
    for (var z: zombies) { z.display(); }
    if (player.health <= 0) { gameOver(); }
  }
}
```

```
//imports
//Sprite and Player can be eliminated, if the student wants to
//treat the player as Zombie nr 0.
public class Sprite {
  public Integer x, y; //x,y is middle of the sprite
  public Integer h, w; //height, weight
  Bitmap picture; //no animation yet, just movement
  void Move(int speed, int direction) {...}
  boolean Overlaps(Sprite s) {...}
}
public class Player extends Sprite {
  final Integer speed = 100; //fixed speed
  public Integer direction; //1..360 degrees
  public Integer health = 100;
}

public class Game {
  final static int maxZombieCount = 10000; //10'000 zombies max
  final static int ZombieKindCount = 20;
  /*static*/ Player player;

  static int ZombieCount;
  static int ZombieKind[maxZombieCount]; //or +1 if player=zombie0
  //static int PlayerHealth; //needed if Player = zombie0
  static int ZombieXYWH[maxZombieCount][4];
  static int ZombieSpeed[ZombieKindCount];
  static int ZombieDir[maxZombieCount];
  static Bitmap[ZombieKindCount];
  static void Move(int ZombieId) {...}
  static int Overlaps(int ZombieId, Player player) {...}
  //if player static, or if player=zombie0 Overlaps can be:
  //static int Overlaps(int ZombieId /*, int playerId*/) {...}

  Game() {...}
  static void step() {
    player.Move(...); //integrate into for loop is player=zombie0
    for (var z = 0; z < maxZombieCount; z++) { Move(z); }
  }

  void doPlayerHit() {
    var oldHealth = player.health;
    for (var z = 0; z < maxZombieCount; z++) {
      player.health -= Overlaps(z, player); }
    if (player.health != oldHealth) { ... }
  }
  void display() { //runs 30x per second
    doPlayerHit();
    player.display();
    for (var z = 0; z < maxZombieCount; z++) {
      display(z);
    }
    if (player.health <= 0) { gameOver(); }
  }
}
```